

Задание №3.

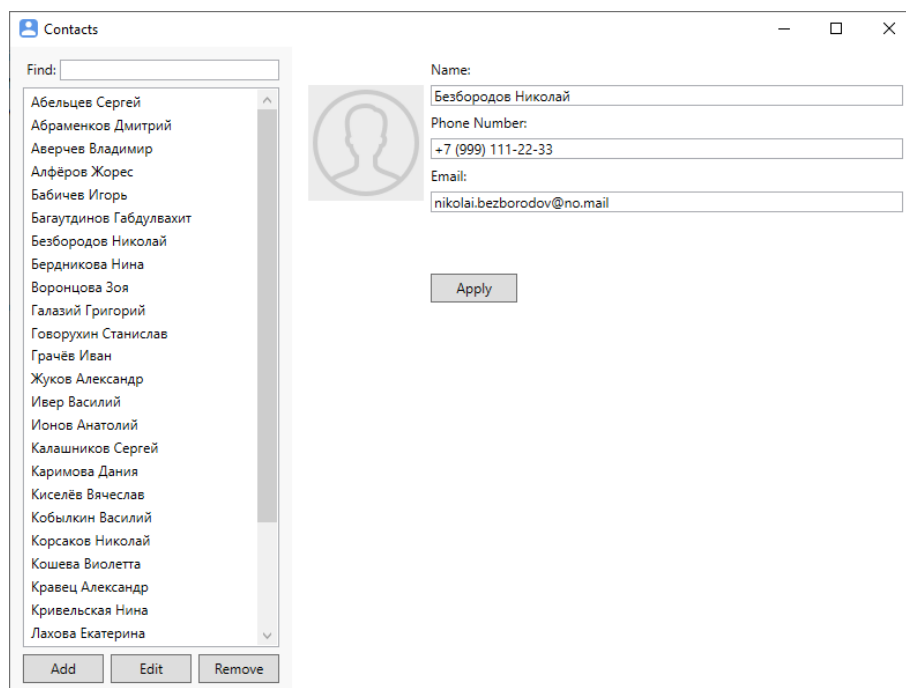
Работа с коллекцией объектов

Цель задания:

- Создать Wpf-приложение, позволяющее выполнять редактирование коллекции объектов.

Приложение

В предыдущем задании мы разработали приложение для редактирования данных об одном объекте, познакомились с концепцией привязки данных. В этом задании нам необходимо разработать приложение, которое позволит просматривать и редактировать коллекцию однотипных объектов:



Приложение представляет собой две панели, разделенных передвигаемым сплиттером (GridSplitter). На левой панели отображается список контактов – объектов ранее созданного класса Contact. На правой панели отображается выбранный в левом списке контакт. Выбранный контакт доступен только на чтение, редактирование заблокировано (свойство IsReadOnly для TextBox). Кнопка Apply по умолчанию скрыта (свойство Visibility).

Add Contact. При нажатии на кнопку Add, в списке слева исчезает выделение текущего элемента, на панели справа все поля становятся пустыми, внизу становится доступна кнопка Apply. Пользователь вводит данные на панели и нажимает на кнопку Apply. После нажатия на кнопку, созданный контакт добавляется в список слева и становится выбранным, кнопка Apply скрывается.

Если во время создания контакта пользователь выберет другой контакт (кликнет на любой контакт в левом списке), то создание контакта должно отмениться – введенные пользователем данные стираются, на правой панели отображается выбранный в списке слева контакт, а кнопка Apply должна скрыться.

Edit Contact. Если в левом списке выбран контакт, то становится доступной кнопка Edit. При нажатии на кнопку Edit, поля на правой панели становятся доступными для редактирования, появляется кнопка Apply. Аналогично кнопке Add, после редактирования данных пользователь должен нажать на Apply, чтобы введенные данные применились. При выборе другого контакта в списке, редактирование завершается без сохранения изменений. После завершения редактирования, кнопка Apply скрывается. Если во время редактирования было изменено имя контакта, то это имя должно обновиться в левом списке.

Если список контактов пустой или в списке ничего не выбрано, то кнопка Edit должна быть заблокирована.

Remove Contact. По нажатию на кнопку Remove происходит удаление выбранного контакта. После удаления контакта из списка, программа должна установить выделение на следующий после удаленного контакт. Если удаленный контакт был последним в списке, то выделение должно установиться на последний контакт после удаления. Если удаленный контакт был единственным в списке, то после удаления выделение в списке убирается, на правой панели отображаются пустые поля. Если список пустой или в списке ничего не выбрано, то кнопка Remove должна быть заблокирована.

Сохранение данных между сессиями. Программа должна сохранять и загружать все введенные данные между запусками программы. Для этого модифицируйте класс ContactSerializer для работы с коллекцией контактов.

Прочая функциональность. Чтобы не создавать не нужных коллизий во время работы приложения, рекомендуется во время редактирования контакта или создания нового блокировать кнопки Add, Edit и Remove до тех пор, пока пользователь не выйдет из режима создания/редактирования.

В данном приложении не надо реализовывать функциональность поиска, сортировку контактов по алфавиту и добавление/редактирование фото контакта. Эти функции будут реализованы позже.

По желанию, для упрощения отладки в программу можно добавить функцию генерации случайных данных.

Требования к верстке. Приложение имеет *адаптивную* двухколоночную верстку, где ширину колонок **можно регулировать** с помощью разделителя (GridSplitter).

Левая колонка имеет светло-серый фоновый цвет (#f8f8f8), вплотную подходит к полям окна. Содержимое колонки имеет отступ в 8 пкс с каждой стороны. Помимо этого, каждый элемент управления в колонке имеет свой отступ в 3 пкс. Кнопки имеют высоту 25 пкс, ширина зависит от ширины колонки.

Правая колонка имеет белый фон, содержимое колонки имеет отступ 8 пкс с каждой стороны. Внутренние элементы управления также имеют собственные отступы в 3 пкс. Плейсхолдер для фото контакта имеет размер 100x100, выровнен по верхней границе текстового поля с именем. Кнопка Apply отстоит от остальных элементов управления на 50 пкс, размеры 75x25.

Передвижной разделитель между колонками имеет ширину 3 пкс, белый цвет.

Требования к реализации. Для привязки (data binding) к коллекциям объектов, необходимо использовать класс ObservableCollection. Аналогично классу List, ObservableCollection предоставляет хранение объектов по индексу, однако дополнительно реализует интерфейс INotifyPropertyChanged. Таким образом, при изменении коллекции (добавлении, удалении, изменении объектов) пользовательский интерфейс автоматически обновится. Чтобы коллекция уведомляла также и об изменении данных внутри хранимых объектов, необходимо, чтобы эти объекты реализовывали интерфейс INotifyPropertyChanged. Это значит, что вам потребуется либо создать класс ContactVM, который будет агрегировать объект класса Contact и реализовывать интерфейс INotifyPropertyChanged, либо реализовать интерфейс INotifyPropertyChanged в самом классе Contact. В зависимости от выбранной реализации MainVM будет хранить свойство типа ObservableCollection<ContactVM> или ObservableCollection<Contact>.

Отдельное свойство в MainVM должно хранить выбранный в списке контакт – это упростит реализацию функциональности.

Для блокировки пользовательских элементов создайте в MainVM вычисляемые булевы свойства, доступные только на get. Элементы пользовательского интерфейса должны быть привязаны к этим булевым свойствам (IsReadOnly, IsEnabled, Visibility). Блокировку кнопок можно реализовать через методы CanExecute в командах.

Для привязки к свойству Visibility у элементов управления, понадобится реализовать так называемый [конвертер значений](#).

Реализация предполагает, что для работы кнопок вы создадите отдельные классы команд. Однако, приветствуется использование единого класса RelayCommand для всех команд, или использование стороннего MVVM-фреймворка.

Задание должно выполняться в отдельной ветке. По завершении реализации должен быть отправлен запрос на вливание (Pull Request) в ветку develop.

Напоминание: при реализации приложения уровень VM не должен ничего знать о сущностях, работающих на уровне View.

Перед выполнением задания необходимо ознакомиться с источниками [1, 2].

Литература

1. **Мак-Дональд М.**
WPF 4: Windows Presentation Foundation в .NET 4.0 с примерами на C# 2010 для профессионалов пер. с англ. – М.: ООО «И.Д. Вильямс», 2011. – 1024 с. : ил. – парал. тит. англ. (главы 1-9)
2. **Metanit.com. Руководство по WPF**
/ Metanit.com. – URL: <https://metanit.com/sharp/wpf/> (главы 1-7, 10)